

PERCEPTION | JUDGMENT | OPTIMIZATION

Modeling Artificial Intelligence Beyond the LLM

A functional taxonomy of models, reasoning, orchestration, control, and autonomous intelligence

The central educational proposition is simple but consequential: **artificial intelligence is not a model class; it is an architecture of heterogeneous computational functions.** The self-driving car provides the laboratory for seeing how perception, prediction, judgment, optimization, control, reasoning, and governance cooperate.

Alejandro Reynoso

Conceptual paper / teaching framework

August 2026

Artificial Intelligence

Reader’s Map

The question this paper asks
If an LLM is only one family of models, **what else must an intelligent system be able to do?** The paper answers by separating the functions of intelligence from the computational machinery used to implement them.

Part	What the reader should understand
1. Functional anatomy	Why perception, prediction, judgment, optimization, and control solve different problems.
2. AI toolbox	Where classical ML, deep learning, generative models, LLMs, reasoning models, RL, optimization, and control belong.
3. Taxonomy	Why model, paradigm, epistemic output, architectural role, and autonomy are different axes.
4. Self-driving laboratory	How an autonomous car progresses from an LLM-less ML stack to deeply embedded hybrid intelligence.
5. Orchestration	Why the next frontier is not generic tool calling but model-aware selection and coordination.
6. Curriculum	What students should master to become architects of AI rather than users of one model family.

The recurring mental model

Observe → Perceive → Predict → Judge → Optimize → Act → Learn.

Different model families can occupy each position. The architecture matters as much as the individual model.

Abstract

The current pedagogical emphasis on large language models risks collapsing the much broader field of artificial intelligence into a single model family. This paper develops a functional and architectural taxonomy of AI intended for advanced teaching and system design. Its central proposition is that intelligence is not adequately characterized by language generation or reasoning alone. Operational intelligence requires at least three irreducible functions: *perception*, which transforms observations into representations of state; *judgment*, which interprets state in context, reasons about objectives, uncertainty, regimes, and competing hypotheses; and *optimization/action*, which selects and executes feasible interventions under constraints. Around this triad sit memory, learning, world modeling, prediction, control, monitoring, and governance.

The paper distinguishes model classes from system roles and develops a comprehensive toolbox spanning classical statistical learning, deep learning, representation and generative models, sequence and state-space models, transformers and foundation models, large language and reasoning models, probabilistic inference, reinforcement learning, mathematical optimization, control, world models, tool-using agents, multi-agent systems, and autonomous architectures. Autonomous driving is used as a deliberately concrete test case: a capable self-driving system can exist without an LLM, while an LLM or multimodal reasoning model can add semantic judgment and orchestration without safely replacing the perception, optimization, and control stack. The resulting framework replaces the coarse contemporary formula “LLM + tools + agents” with a model-aware architecture in which an orchestrator understands the epistemic role, uncertainty, authority, and failure modes of the capabilities it coordinates. The educational implication is a course in *Modeling Artificial Intelligence*: students should learn not merely individual models, but how heterogeneous forms of intelligence are composed into reliable autonomous systems.

Introduction: The Mistake of Equating AI with the LLM

The success of large language models has produced a remarkable inversion in the teaching of artificial intelligence. Historically, AI and machine learning courses moved from statistical estimation and classification toward representation learning, neural networks, sequential models, reinforcement learning, planning, and control. Today, many students encounter a conversational foundation model first and infer that the remaining machinery is either subordinate or obsolete.

That inference is conceptually dangerous. Deep learning itself was important precisely because layered models could learn increasingly abstract representations from raw data [1]. Transformers subsequently changed sequence modeling by replacing recurrence with attention-based architectures [2]. Variational autoencoders provided a framework for learned latent-variable inference [3]. Deep reinforcement learning demonstrated learned control from high-dimensional observations [4]. None of these problems—representation, temporal structure, latent inference, sequential decision, or control—ceases to exist because an LLM can discuss them.

A better starting identity is:

Artificial Intelligence is a system property, not a synonym for one model class.

The fundamental educational question therefore changes from “Which model is fashionable?” to:

What functions must intelligence perform, and which models can perform them reliably?

Research Question, Scope, and Contributions

The paper addresses a specific conceptual problem: contemporary AI education and system design often mix categories that belong to different levels of analysis. A CNN, reinforcement learning, an LLM, an optimizer, an agent, and an autonomous system may all be described casually as “AI models,” even though one is a neural architecture, another a learning paradigm, another a foundation-model class, another a decision procedure, another a software/agency pattern, and another a system-level property.

The research question is therefore:

Can artificial intelligence be represented by a common architecture that distinguishes the functions intelligence must perform from the model families, learning paradigms, epistemic outputs, system roles, and authority structures used to implement them?

The paper makes five claims.

1. **Functional claim.** Operational intelligence requires a closed loop that includes perception/state construction, anticipation, judgment, optimization/decision, execution, and feedback. Language competence alone is neither necessary nor sufficient for this loop.
2. **Taxonomic claim.** AI should be classified along multiple axes rather than a single historical sequence of algorithms. Model family, learning paradigm, function, architecture, epistemic output, and autonomy are different dimensions.
3. **Orchestration claim.** The progression from tool-using agents to mature autonomous systems requires *model-aware orchestration*: the coordinating layer should know not only which tool is callable, but what kind of model it is, what its output means, how uncertainty is represented, and what authority it possesses.
4. **Safety claim.** Intelligence and authority must be separated. A component may possess superior semantic or reasoning capability while remaining subordinate to deterministic constraints, formal verification, or a safety veto.
5. **Pedagogical claim.** A rigorous AI curriculum should train students to compose heterogeneous intelligences, not merely to operate a dominant model family.

The framework is intentionally architectural rather than neuroscientific. “Perception,” “judgment,” and “optimization” are engineering abstractions, not claims that human cognition decomposes cleanly into three biological modules. Likewise, the taxonomy does not assert that particular model families have immutable roles. Modern foundation models increasingly cross functional boundaries. The purpose is to make those crossings explicit rather than to deny them.

A Functional Theory: Perception, Judgment, and Optimization

For operational systems, three functions provide a useful irreducible core:

$$\mathcal{I} = \mathcal{P} + \mathcal{J} + \mathcal{O}$$

where $\mathcal{P}$ denotes perception and state representation, $\mathcal{J}$ denotes judgment and contextual reasoning, and $\mathcal{O}$ denotes optimization, decision, and action. This is not intended as a psychometric definition

of intelligence. It is an engineering decomposition of the functions required for an artificial system to operate in an uncertain environment.

Perception: constructing an actionable state

An intelligent system never receives “reality” directly. It receives measurements:

$$y_t = h(x_t) + \epsilon_t,$$

where x_t is the latent state of the world, y_t is an observation, $h(\cdot)$ is a measurement process, and ϵ_t represents noise.

Perception is therefore an inference problem:

$$p(x_t \mid y_{1:t}).$$

For a vehicle, y_t may include pixels, radar returns, LiDAR point clouds, GPS, and inertial measurements. For another domain it may be transactions, text, prices, laboratory measurements, or telemetry. Intelligence begins by transforming high-dimensional, incomplete observations into a representation sufficiently structured for downstream reasoning.

Perception includes detection, classification, segmentation, dimensionality reduction, representation learning, sensor fusion, localization, latent-state estimation, and anomaly detection. Deep learning has been particularly successful because learned representations can organize data into multiple levels of abstraction [1].

Prediction: extending perception through time

A state estimate is incomplete unless the system can model dynamics:

$$p(x_{t+1:t+H} \mid x_{\leq t}, a_{\leq t}).$$

Prediction is the bridge between perception and judgment. It introduces temporal models, recurrent networks, LSTMs, transformers, state-space models, survival models, Bayesian forecasting, ensembles, and simulation.

A pedestrian detector answers “what is there?” A trajectory model asks “where might it be?” A risk model asks “with what probability and uncertainty?” These are distinct computational questions.

Judgment: assigning meaning, relevance, and regime

Judgment begins where prediction alone becomes insufficient. Consider:

$$P(\text{pedestrian enters lane}) = 0.30.$$

The number does not determine behavior. Thirty percent may be tolerable in one decision problem and catastrophic in another. Context matters: school zone, visibility, vehicle speed, pedestrian age, escape alternatives, and the asymmetry of errors.

Judgment maps state and predicted futures into a contextual interpretation:

$$J_t = g(\hat{x}_t, \mathcal{F}_t, \mathcal{K}, G, C),$$

where $\mathcal{F}_t$ denotes forecast distributions, $\mathcal{K}$ knowledge, G goals, and C contextual constraints.

This is the domain in which foundation models, multimodal models, LLMs, and large reasoning models become particularly important. Reasoning-oriented models increasingly allocate inference-time computation to multi-step deliberation, but recent surveys also emphasize the cost and inefficiency of excessively long reasoning traces [15]. Judgment is therefore not synonymous with “more tokens.” It includes contextual interpretation, hypothesis comparison, regime recognition, semantic abstraction, goal reconciliation, and meta-decisions about which specialized model should be consulted.

Optimization: converting judgment into feasible action

Judgment defines what matters; optimization selects what to do:

$$a_t^* = \arg \min_{a \in \mathcal{A}(x_t)} \mathbb{E} [L(a, X_{t+1:t+H}) \mid \mathcal{F}_t, J_t],$$

subject to

$$g_i(a, x_t) \leq 0, \quad h_j(a, x_t) = 0.$$

The loss function can encode safety, efficiency, comfort, cost, progress, fairness, or other objectives. Constraints encode what cannot be violated.

This is the domain of mathematical programming, operations research, dynamic programming, search, reinforcement learning, model-predictive control, game theory, and robust/stochastic optimization.

The distinction is fundamental:

Judgment specifies or modifies the decision regime; optimization solves within it.

Action, control, and feedback

A selected policy still must be executed:

$$u_t = \pi(x_t, r_t),$$

followed by a new observation:

$$x_t \rightarrow u_t \rightarrow x_{t+1} \rightarrow y_{t+1}.$$

Operational intelligence is therefore closed-loop:

Perceive $\rightarrow$ Predict $\rightarrow$ Judge $\rightarrow$ Optimize $\rightarrow$ Act $\rightarrow$ Observe Again.

Why the Three Functions Are Jointly Necessary

Table 1 shows why none of the three dimensions is a substitute for the others.

Thus a fluent model with weak state estimation is not operationally intelligent in the same sense as a closed-loop autonomous system. Conversely, an optimizer can be mathematically perfect and still optimize an incorrectly perceived state or an inappropriate objective.

Necessary Functions, Substitutable Implementations

Table 1: The three core functions of operational intelligence

Function	Core question	Without it	Representative machinery
Perception	What is the state?	The system reasons about an inaccurate or unstructured world.	Vision, classifiers, embeddings, sensor fusion, state estimation, autoencoders, probabilistic inference.
Prediction	What may happen?	Decisions are reactive rather than anticipatory.	LSTMs, transformers, state-space models, world models, simulation, Bayesian forecasting.
Judgment	What does it mean and what matters?	The system may optimize the wrong objective or apply the wrong regime.	LLMs, reasoning models, multimodal models, knowledge systems, causal/semantic reasoning, orchestration.
Optimization	What should be done?	Insight does not become a disciplined choice among alternatives.	Search, mathematical programming, dynamic programming, RL, MPC, game theory.
Control	How is it executed?	High-level decisions do not become stable, timely actions.	Feedback control, PID, MPC, safety controllers, deterministic execution.

A central distinction is between *functions* and *implementations*. The paper treats the functional loop as relatively persistent while allowing the implementation to change radically.

Let an operational task require the functional set

$$\mathcal{F}^* = \{P, F, J, O, C\},$$

where P is perception/state estimation, F forecasting, J judgment, O optimization/decision, and C control/execution.

For each function $f \in \mathcal{F}^*$, let

$$\Gamma(f) = \{\gamma_1, \gamma_2, \dots\}$$

denote the set of model families capable of implementing it. Technological progress changes $\Gamma(f)$ and the relative performance of its members; it does not necessarily eliminate the function itself.

This resolves a common pedagogical confusion. If transformers replace LSTMs in a forecasting task, *temporal modeling has not disappeared*. If a multimodal foundation model replaces a specialized CNN in part of a perception stack, *perception has not disappeared*. If an LLM writes an optimization model, *optimization has not disappeared*. The implementation boundary moved.

The resulting principle is:

Models are substitutable; computational functions are more persistent.

Pedagogical checkpoint

Do not ask first, “Which model should I use?” Ask first, “What computational function must be performed?” A new model can replace an implementation without eliminating the underlying function.

Table 2: Persistent function versus substitutable implementation

Persistent problem	Possible implementations	What technological substitution does not remove
State estimation	Filters, CNNs, ViTs, multimodal models, Bayesian inference	Need to distinguish observation from latent state
Temporal prediction	ARIMA/state space, LSTM, transformer, world model	Need to reason about future states and uncertainty
Semantic judgment	Rules, expert systems, LLMs, LRMs, hybrid knowledge systems	Need to determine relevance, context, and regime
Decision	Rules, search, RL, optimization, LLM-assisted planning	Need to compare feasible alternatives
Execution	Scripts, APIs, controllers, robotic actuators	Need for reliable state-changing action
Safety/governance	Hard constraints, formal methods, critics, human approval	Need to bound authority and failure consequences

The AI Toolbox: Models, Paradigms, and Their Intellectual Roles

The relevant taxonomy should not be “old ML versus new AI.” The toolbox is layered.

Table 3: A functional taxonomy of the AI toolbox

Family	Representative methods	Primary contribution	Characteristic limitation
Classical statistical learning	Linear/logistic models, trees, random forests, boosting, SVMs, clustering	Estimation, classification, interpretable structure, tabular prediction	Often depends on engineered features; limited unstructured representation capacity
Deep discriminative learning	CNNs, deep MLPs, ViTs	Learned high-dimensional perception and classification	Data hungry; uncertainty and causal meaning may remain weak
Representation learning	Autoencoders, VAEs, embeddings, contrastive learning	Compression, latent structure, anomaly detection, learned state spaces	Latent variables need not correspond to causal or decision-relevant concepts
Sequential models	RNNs, LSTMs, temporal CNNs, transformers, state-space models	Temporal dependence, sequence prediction, long-horizon context	Forecast errors compound; distribution shift matters
Probabilistic models	Bayesian networks, HMMs, Gaussian processes, filters, ensembles	Explicit uncertainty, latent-state inference, belief updating	Modeling assumptions and computational complexity
Generative models	VAEs, GANs, diffusion models, autoregressive models	Scenario generation, simulation, density/latent modeling, synthetic data	Plausibility is not equivalent to correctness or safety
Foundation models	Language, vision-language, multimodal pretrained models	Transfer, semantic abstraction, cross-task interfaces	Calibration, grounding, latency, hallucination, controllability

Family	Representative methods	Primary contribution	Characteristic limitation
Large reasoning models	Deliberative/test-time-compute models	Multi-step reasoning, search-like deliberation, difficult symbolic/semantic tasks	Cost, variable reliability, excessive reasoning, verification requirements
Knowledge systems	Retrieval, knowledge graphs, ontologies, RAG	Grounding, institutional memory, explicit relations	Retrieval quality and ontology maintenance constrain performance
Reinforcement learning	Value methods, policy gradients, actor-critic, model-based RL	Sequential decisions and learned policies from reward	Reward specification, exploration, safety, sample efficiency
Optimization/OR	LP, QP, NLP, stochastic/robust optimization, combinatorial search	Feasible decisions under explicit objectives and constraints	Requires explicit formulation; model misspecification
Control	PID, LQR, MPC, robust/adaptive control	Stable low-latency execution in dynamical systems	Depends on state/model quality and specified envelopes
World models	Learned latent dynamics, predictive simulators, occupancy/world representations	Internal simulation of environment and action consequences	Model error can produce systematically wrong imagined futures
Tool-using agents	ReAct, function calling, Toolformer-style architectures	Connect reasoning to external computation and information	Tool selection, error propagation, authority and verification
Autonomous agents	Memory, planning, reflection, tool use, persistent goals	Sustained multi-step action over time	Long-horizon drift, compounding errors, governance and containment
Multi-agent systems	Specialized agents, debate, delegation, coordination	Decomposition, specialization, parallel search, cross-checking	Coordination cost, emergent behavior, correlated failure

The important pedagogical point is that these families are *not substitutes*. A system may simultaneously require a vision transformer, Bayesian state estimator, LLM, constrained optimizer, and deterministic controller.

Models Versus Paradigms Versus Architectures

Students frequently confuse three levels of abstraction.

A transformer is a model architecture. Self-supervised pretraining is a learning paradigm. An LLM orchestrating perception, retrieval, optimization, and control is a system architecture. Failure to separate these levels produces the superficial statement that “the LLM does everything.”

Canonical Architecture Patterns

Table 4: Three levels that should not be conflated

	Definition	Examples	Question answered
Model	A parameterized computational mapping or inference mechanism	CNN, LSTM, VAE, transformer, Bayesian network	“How is this particular inference computed?”
Paradigm	A general way of learning or deciding	Supervised, unsupervised, self-supervised, generative, RL, optimization, planning	“What learning/decision principle is being used?”
Architecture	Organization of multiple components, information flows, authority, memory, and feedback	Perception–planning–control stack; RAG agent; multi-agent hierarchy	“How do multiple capabilities become a system?”

The taxonomy becomes more useful when mapped into recurrent system designs. Five patterns are especially important.

Table 5: Canonical architectures of artificial intelligence

Pattern	Information flow	Interpretation
Pipeline	$M_1 \rightarrow M_2 \rightarrow M_3$	Fixed sequence of specialized models; simple and auditable, but brittle to unanticipated routing needs
Mixture/gating	$x \rightarrow G(x) \rightarrow M_k$	Learned or rule-based gate selects an expert; precursor to model routing
Planner–executor	Planner $\rightarrow$ task sequence $\rightarrow$ executors	Separates deliberation from action; common in agents and robotics
Orchestrator–specialists	Orchestrator $\leftrightarrow \{M_1, \dots, M_n\}$	Dynamic decomposition and model selection; natural role for LLM/LRM but not restricted to them
Hierarchical safety architecture	Orchestrator $\rightarrow$ optimizer $\rightarrow$ safety veto $\rightarrow$ controller	Separates semantic intelligence, prescriptive computation, authority, and execution

These patterns also clarify the historical progression. A conventional ML pipeline may already contain several forms of intelligence but no semantic orchestrator. An agentic architecture adds dynamic routing and planning. A mature autonomous architecture adds persistent state, explicit authority, feedback, model selection, monitoring, and recovery.

The orchestrator is a meta-model

An orchestrator can itself be represented as a policy over models:

$$\Omega(q, s) = P(\gamma_i, \rho_i \mid q, s),$$

where q is the current problem, s the system state, γ_i a candidate capability, and ρ_i an assigned role or authority.

This makes orchestration a *meta-level decision problem*. The system is not only deciding what action to take; it is deciding which computational machinery should participate in producing that action.

Pedagogical checkpoint

The autonomous car is used as a controlled thought experiment. Hold the driving task constant and progressively change the architecture. This isolates what an LLM actually adds instead of attributing the entire system’s intelligence to the most visible model.

The Autonomous-Car Laboratory: From ML-Only Autonomy to Deeply Embedded LLM Intelligence

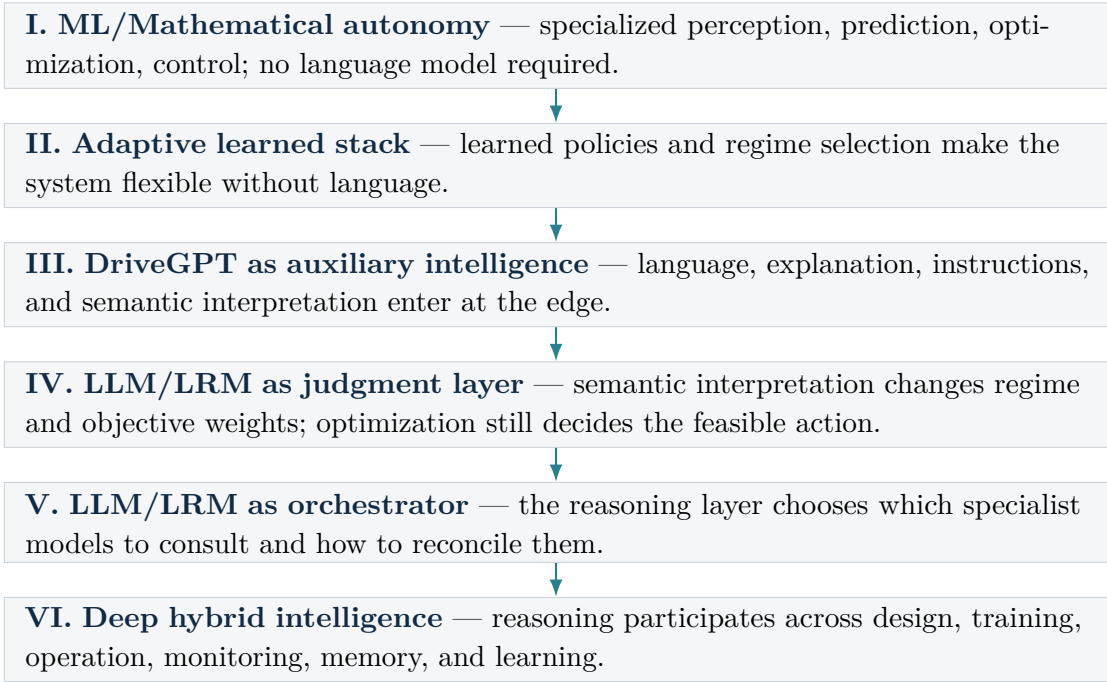
Autonomous driving provides a controlled conceptual laboratory in which the architecture of artificial intelligence can be varied while the underlying task remains approximately constant. The vehicle must perceive an uncertain physical environment, anticipate other actors, choose safe trajectories, execute them under dynamical constraints, and continuously revise its state estimate. What changes across architectural generations is where intelligence is located, how heterogeneous models communicate, and what role semantic and reasoning models are permitted to play.

These stages are architectural thought experiments, not a literal description of any one commercial platform. Their purpose is to isolate the marginal contribution of progressively deeper language/foundation-model integration.



The central proposition is not that each later architecture replaces the earlier one. Lower layers remain indispensable. Successive architectures add semantic interpretation, regime formation, model selection, coordination, memory, and learning while retaining specialized machinery where it has comparative advantage.

The architecture ladder at a glance



What changes across the ladder? Not the need for perception, prediction, optimization, or control. What changes is the amount of **semantic judgment, dynamic model selection, orchestration, memory, and lifecycle learning** embedded in the system.

Architecture I: The LLM-less ML/Mathematical Autonomous Car

Begin with the strongest counterexample to the equation $AI = LLM$: a capable autonomous car can contain no language model at all.

$$Y_t \rightarrow \boxed{\text{Perception}} \rightarrow \boxed{\text{State Estimation}} \rightarrow \boxed{\text{Prediction}} \rightarrow \boxed{\text{Planning/Optimization}} \rightarrow \boxed{\text{Control}} \rightarrow U_t.$$

This vehicle is not merely hard-coded. Its perception may be learned; its state estimate probabilistic; its trajectory forecasts neural; its planner optimization-based; and parts of its policy learned through reinforcement learning. It can therefore exhibit substantial adaptive operational intelligence without manipulating natural-language tokens.

Table 6: Architecture I: representative LLM-less autonomous-driving stack

Function	Possible machinery	Output
Sensing	Cameras, LiDAR, radar, GPS, IMU	Raw observations
Perception	CNNs, ViTs, detection, segmentation, sensor fusion	Objects, lanes, free space
Localization/state	SLAM, filters, HD maps, state-space estimation	Vehicle/world state
Prediction	LSTM/transformer trajectory models, probabilistic models	Future trajectories and uncertainty
Planning	Search, mathematical optimization, RL, cost functions	Candidate maneuver/trajectory
Control	MPC, PID, feedback control	Steering, throttle, brake
Safety	Rules, constraints, redundancy, fail-safe controllers	Permit/reject/fallback

The first result is therefore:

Autonomous operational intelligence does not require language intelligence.

Why this matters. This is the baseline against which the incremental contribution of DriveGPT should be measured. If a capability already exists in the LLM-less architecture, it should not be attributed to the LLM merely because the LLM is the most visible component.

Architecture II: From Fixed Rules to Adaptive Learned Driving

The second architecture makes explicit that the alternative to an LLM is not necessarily a brittle rule engine. There is a long continuum:

rules $\rightarrow$ statistical inference $\rightarrow$ deep perception $\rightarrow$ learned prediction $\rightarrow$ optimization $\rightarrow$ RL/adaptive policies.

Driving regimes can already be represented as

$$\Pi = \{\pi_{normal}, \pi_{defensive}, \pi_{construction}, \pi_{emergency}, \pi_{comfort}\}.$$

A non-language system can select among them using thresholds, classifiers, probabilistic state machines, learned gates, or optimization. Thus flexible regime selection is not uniquely an LLM

capability. This establishes the proper counterfactual against which the incremental value of an LLM should be measured.

Architecture III: DriveGPT as an Auxiliary Semantic Intelligence

Now preserve the validated ML/math stack and add an LLM or multimodal foundation model at the periphery. DriveGPT does not steer. It creates a semantic interface between humans, contextual information, and the existing machine architecture.

Its contributions can include: (i) translating natural-language intentions into destinations and constraints; (ii) interpreting temporary signs, gestures, and unusual combinations of recognized objects; (iii) explaining why the system acted; (iv) retrieving relevant rules or prior cases; and (v) asking clarifying questions when passenger intent is ambiguous.

The principle is:

LLM adds semantics without yet adding operational sovereignty.

Architecture IV: DriveGPT as Judgment and Regime Selection

The next architecture moves semantic intelligence into the decision loop. Suppose the lower stack reports

$$z_t = \{\text{school zone, occluded pedestrian, } P(\text{cross}) = .31, \text{ wet road, reduced visibility}\}.$$

A numerical system can map these variables into a pre-specified policy. A multimodal LLM or reasoning model can additionally form a contextual judgment: the conjunction of school zone, occlusion, and reduced visibility warrants a defensive regime even though no single statistic is extreme.

Formally,

$$J_t = g(B_t, W_t, K_t, G_t), \quad J_t \rightarrow r_t,$$

and judgment can modify optimizer weights:

$$L_t(a) = \lambda_{S,t} R_{safety}(a) + \lambda_{P,t} R_{progress}(a) + \lambda_{C,t} R_{comfort}(a).$$

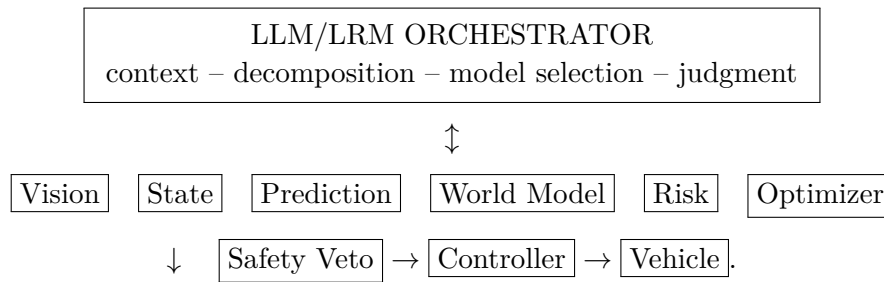
A defensive judgment can imply $\lambda_{S,t} \uparrow$ and $\lambda_{P,t} \downarrow$. The optimizer still computes the feasible trajectory and the controller still executes it. Judgment changes the decision regime; it does not hallucinate brake pressure.

Architecture V: The LLM/LRM as Orchestrator of Specialized Intelligence

The next progression is qualitatively different. The reasoning model participates in deciding which computational capabilities should be used. Let

$$\Gamma = \{\gamma_{vision}, \gamma_{localization}, \gamma_{trajectory}, \gamma_{risk}, \gamma_{map}, \gamma_{world}, \gamma_{optimizer}, \gamma_{controller}\}.$$

The orchestrator can ask whether perception is sufficiently certain, request another sensor analysis, choose a trajectory model, ask a world model for counterfactual futures, reconcile specialist disagreement, change the risk regime, or escalate to a human.



This is the transition from generic tool calling to *model-aware orchestration*. The orchestrator should know that a trajectory tool returns a forecast distribution, that an optimizer returns a solution conditional on objectives and constraints, that a semantic hypothesis requires grounding, and that the safety controller has veto authority. It becomes executive cognition over a federation of specialized intelligences.

Architecture VI: Deeply Embedded Hybrid Autonomous Intelligence

The deepest integration occurs when language/reasoning models cease to be one box in the operational stack and participate across the lifecycle:

Design → Train → Test → Operate → Monitor → Learn → Redesign

At design, they can translate requirements, inspect architectures, and generate edge cases. During training they can assist data curation, semantic labeling, synthetic scenarios, and failure taxonomy. During testing they can generate unusual scenarios and synthesize evidence. During operation they can contribute judgment, model selection, interaction, and explanation. During monitoring they can connect patterns across logs. During learning they can convert episodes into structured institutional knowledge and new tests or policies.

The system is no longer well described as LLM + car. It is a hybrid cognitive architecture:

Foundation/Reasoning Models + Specialized ML + World Models + Optimization + Control + Memory + Governance

Comparative anatomy of the six architectures

Table 7: Progression from ML-only driving to deeply embedded hybrid intelligence

Architecture	Dominant intelligence	Incremental capability	What remains specialized / constrained
I. ML/Math	Perception, prediction, optimization, control	Operational autonomy without language	Entire stack; no LLM
II. Adaptive ML	Learned policies/regimes	Flexibility beyond fixed rules	Semantic/language reasoning absent
III. DriveGPT auxiliary	Semantic/language intelligence	Instructions, explanation, contextual interpretation	Planning, safety, control
IV. Judgment layer	Context/regime intelligence	Objective and policy-regime configuration	Numerical optimization and physical control
V. Orchestrator	Meta-level architectural intelligence	Model selection, decomposition, reconciliation	Safety authority and low-level execution
VI. Deep integration	Lifecycle/system intelligence	Reasoning across design, operation, monitoring, learning	Hard constraints and specialized high-frequency computation

One construction-zone event across all six architectures

Consider a temporary construction zone. A worker in reflective clothing gestures vehicles into a lane that would ordinarily be inappropriate; temporary cones conflict partially with permanent lane

markings.

Table 8: The same event under progressively richer architectures

Architecture	Response
I	Specialized models detect worker, cones, lane boundaries, and vehicles; prediction and optimization determine whether a validated feasible path exists. The system may stop if the state lies outside its operating envelope.
II	Learned gating recognizes a construction-like regime and activates a conservative policy without language, but interpretation remains bounded by trained categories and engineered variables.
III	DriveGPT interprets semantic context and can explain that a worker appears to be temporarily directing traffic; the classical stack still determines whether movement is feasible.
IV	Judgment classifies a temporary traffic-control regime, increases safety/uncertainty weights, and reduces progress priority; the optimizer solves the modified problem.
V	The orchestrator requests a second gesture analysis, checks map/context information, asks a world model to simulate alternatives, compares prediction outputs, and then asks the planner to evaluate trajectories consistent with the inferred regime. Safety retains veto power.
VI	The event is also stored semantically. Recurrent cases can be clustered, converted into new simulations, tests, model updates, policies, and institutional knowledge.

This worked example shows why the distinctive contribution of the LLM is not simply better pixel recognition or faster control. As integration deepens, it contributes semantic interpretation, objective formation, model orchestration, and eventually lifecycle learning.

The direct-LLM-driving counterfactual

One can imagine an end-to-end architecture:

Sensors $\rightarrow$ Multimodal Foundation Model $\rightarrow$ Steering/Brake.

Increasingly unified end-to-end and world-model approaches make this an important research direction. But the burden of proof changes rather than disappears. A safety-critical controller must still satisfy latency, stability, calibration, constraint satisfaction, failure detection, and verification requirements. If one learned model internalizes perception, prediction, planning, and control, those *functions* remain analytically present even when their implementation boundaries disappear.

Thus end-to-end learning does not invalidate the taxonomy. It compresses several functional roles into a common learned representation.

The architectural frontier: degree of semantic integration

Let $\eta \in [0, 1]$ represent conceptually the degree to which foundation/reasoning-model cognition penetrates the autonomous loop. At low η , the LLM is an interface. At intermediate η , it contributes judgment and orchestration. At high η , foundation models share representations with perception,

prediction, planning, memory, and learning.

The trade-off is important:

$$\text{semantic flexibility} \uparrow \iff \text{verification complexity} \uparrow,$$

and potentially

$$\text{integration} \uparrow \iff \text{component-level auditability} \downarrow.$$

The optimal architecture is therefore not necessarily the one with maximum LLM penetration. It is the one that allocates each function to the mechanism with the best combination of capability, reliability, latency, cost, auditability, and safety.

Four conclusions from the autonomous-car laboratory

The progression yields four propositions:

AI can be highly autonomous without an LLM.

Adding an LLM need not improve functions already solved well by specialists.

LLM/LRM comparative advantage increasingly lies in semantics, judgment, decomposition, and orchestration.

Deep integration creates hybrid intelligence, not simple LLM replacement of traditional AI.

The self-driving car therefore provides a concrete demonstration of the paper’s broader thesis: advanced artificial intelligence is best understood as an architecture in which heterogeneous computational forms cooperate, with different models responsible for different kinds of inference and different levels of authority.

From Tools to Epistemically Typed Capabilities

Tool-use research already demonstrates the value of letting language models call external computation. Toolformer explicitly learns which API to call, when, and with what arguments [8]; ReAct interleaves reasoning with actions that obtain new information [7]. Multi-agent frameworks such as AutoGen allow combinations of LLMs, human inputs, and tools [9]. Surveys of LLM-agent planning classify mechanisms such as task decomposition, plan selection, external modules, reflection, and memory [11].

The next conceptual step is to stop treating every external capability as an untyped “tool.”

This is **model-aware orchestration**. The orchestrator reasons about the epistemic type of an output, not merely its textual representation.

A Hierarchy of Artificial Intelligence Architectures

The progression can be represented as six architectural stages.

The progression is not necessarily chronological. Safety-critical systems may deliberately retain Stage 0 components inside Stage 5 architectures because deterministic constraints can be desirable.

Table 9: From generic tool calling to model-aware orchestration

Capability	What the orchestrator should know	Typical output
Classifier	Classes, calibration, threshold, domain of validity	Probabilities / labels
Bayesian estimator	Priors, likelihood, posterior uncertainty	Distribution / credible interval
Forecast model	Horizon, conditioning information, error distribution	Future trajectories / scenarios
LLM/reasoning model	Context window, grounding, verification needs	Semantic hypothesis / plan / explanation
Optimizer	Objective, constraints, feasibility, optimality assumptions	Candidate decision / solution
Simulator/world model	State assumptions and model error	Counterfactual trajectories
Controller	Latency, stability envelope, state inputs	Executable action
Safety system	Non-negotiable rules and veto authority	Permit / reject / fallback

Table 10: Architectural progression from automation to autonomous intelligence

Stage	Architecture	Defining property	Primary limitation
0	Rule-based automation	Explicit IF–THEN logic and deterministic workflows	Brittle outside specified states
1	Statistical/ML pipeline	Learned perception and prediction	Components connected by fixed architecture
2	Optimization/control system	Adaptive decisions under explicit objectives and dynamics	Semantic/contextual interpretation limited
3	Foundation-model augmented	LLM/VLM adds language, semantics, explanation	Often remains an auxiliary layer
4	Agentic orchestration	LLM plans and invokes tools/agents dynamically	Tool taxonomy and authority may be crude
5	Model-aware autonomous architecture	Orchestrator selects heterogeneous models by function, uncertainty, authority, and context	Governance, verification, emergence, long-horizon safety

Autonomous Intelligence Is More Than Agentic Tool Use

An autonomous architecture requires persistence. A useful state description is:

$$S_t = (B_t, M_t, G_t, \Pi_t, \mathcal{T}_t, A_t),$$

where B_t is the current belief state, M_t memory, G_t goals, Π_t policies, $\mathcal{T}_t$ available tools/models, and A_t authority.

An autonomous system repeatedly:

$$S_t \rightarrow \text{observe} \rightarrow \text{update beliefs} \rightarrow \text{plan} \rightarrow \text{invoke models/tools} \rightarrow \text{act} \rightarrow S_{t+1}.$$

Generative-agent research has demonstrated architectures in which observation, memory, reflection, and planning jointly support persistent behavior [10]. Research on computer-using multimodal agents similarly emphasizes memory, exploration, reflection, and safety in realistic environments [12].

The decisive step from “agent” to “autonomous intelligence” is therefore not merely additional tool calls. It is sustained closed-loop agency under changing state, memory, objectives, permissions, and consequences.

Authority Must Be Separated from Intelligence

A highly capable orchestrator should not automatically possess unrestricted authority.

Let $\mathcal{A}_J$ denote actions proposed by the judgment/orchestration layer and $\mathcal{A}_S$ the safe action set. Execution should satisfy:

$$a_t \in \mathcal{A}_J \cap \mathcal{A}_S.$$

If

$$\mathcal{A}_J \cap \mathcal{A}_S = \emptyset,$$

the safety system must reject the proposal and invoke a fallback.

This produces a general principle:

Intelligence, judgment, and authority are separate design variables.

A model may be allowed to recommend, another to optimize, another to verify, and only a constrained controller to execute.

Evaluation: How the Framework Could Be Tested

A taxonomy becomes scientifically useful when it supports comparison and falsification. The proposed architecture suggests several empirical tests.

Let baseline system B be an LLM-less architecture and hybrid system H add a semantic/orchestration layer while preserving the same safety-critical lower stack. Define:

$$\Delta V = V_H - V_B,$$

where V is a vector rather than a single score:

$$V = (S, N, C, E, L, R),$$

with safety S , novel-scenario performance N , contextual correctness C , explanation quality E , latency L , and resource cost R .

The hypothesis is not that H dominates B everywhere. Rather:

$$\Delta N > 0, \quad \Delta C > 0, \quad \Delta E > 0$$

may coexist with

$$\Delta L < 0, \quad \Delta R < 0$$

when lower latency and resource consumption are defined as preferable.

This immediately creates a research program.

The framework is therefore not merely classificatory. It generates hypotheses about where general-purpose reasoning should and should not add value.

Failure Modes of Hybrid Intelligence

Table 11: Illustrative hypotheses generated by the framework

Hyp.	Proposition	Possible test
H1	Specialized models outperform general reasoning models on bounded, high-frequency perception/control tasks at equal latency budgets.	Compare calibrated task accuracy and latency under fixed compute
H2	LLM/LRM augmentation adds most value under semantic ambiguity and regime novelty.	Construct matched conventional versus unusual scenarios
H3	Model-aware routing outperforms generic tool routing when tools have heterogeneous uncertainty and failure modes.	Compare routing policies with/without epistemic meta-data
H4	Separating judgment from execution reduces catastrophic action errors relative to direct LLM action.	Run identical scenarios with and without safety/optimization interposition
H5	Explicit epistemic typing improves multi-model synthesis and calibration.	Measure confidence and decision quality when output types are exposed versus hidden
H6	Increasing orchestrator autonomy without matched governance produces nonlinear growth in long-horizon failure.	Vary autonomy/authority while holding component models fixed

Hybrid architectures introduce failures that are different from failures of individual models.

Table 12: Failure modes by architectural layer

Layer	Failure	Architectural response
Perception	Missed detection, distribution shift, sensor conflict	Redundancy, calibration, sensor fusion, anomaly detection
Prediction	Overconfident trajectory or world-model error	Ensembles, uncertainty bounds, conservative planning
Judgment	Hallucinated context, wrong regime, spurious analogy	Retrieval/grounding, critics, confidence thresholds, human escalation
Orchestration	Wrong model selected; correlated agents; recursive delegation	Typed capabilities, routing constraints, provenance, budgets
Optimization	Wrong objective or omitted constraint	Formal objective review, constraint validation, robust optimization
Control	Instability, actuator mismatch, latency	Certified controller, fail-safe mode, runtime monitoring
Governance	Excess authority, unclear responsibility, unsafe escalation	Least privilege, veto layers, audit logs, explicit authority graph
Learning	Feedback contamination or reward hacking	Offline validation, protected baselines, controlled updates

The key point is that system risk is not reducible to model risk:

$$\text{System Risk} \neq \sum_i \text{Model Risk}_i.$$

Interfaces, delegation, correlated errors, authority, feedback, and emergent interactions create architectural risk.

Pedagogical checkpoint

At this point the student should be able to look at a system and separate six questions: What function is required? Which model implements it? How was it learned? What kind of output does it produce? What role does it play? What authority does it have?

A Comprehensive Curriculum: Modeling Artificial Intelligence

The course should teach the field as an integrated architecture rather than a sequence of fashionable algorithms.

Table 13: Proposed curriculum structure

Block	Theme	Core material	Architectural question
I	Statistical foundations	Probability, estimation, regression, classification, uncertainty, calibration	What can be inferred from data, and with what confidence?
II	Machine learning	Trees, boosting, SVMs, clustering, supervised/unsupervised learning	How are predictive relationships learned?
III	Deep representation	Neural nets, CNNs, embeddings, autoencoders, VAEs, contrastive learning	How is high-dimensional reality represented?
IV	Time and dynamics	RNN/LSTM, transformers, state-space models, filters, forecasting	How does intelligence represent change through time?
V	Generative/world models	Diffusion, autoregression, latent dynamics, simulation	How can the system generate or imagine possible worlds?
VI	Foundation models	Transformers, LLMs, VLMs, multimodal models, RAG	How does general semantic competence enter the architecture?
VII	Reasoning	Deliberation, test-time compute, search, verification, decomposition	When is explicit reasoning useful and how is it checked?
VIII	Decision science	Optimization, OR, dynamic programming, game theory	How are objectives converted into choices?
IX	Reinforcement learning	MDPs, value/policy methods, model-based RL	How can policies be learned from interaction?
X	Control	Feedback, stability, MPC, robust control	How are decisions executed in dynamic environments?
XI	Agents	Memory, planning, reflection, tools, skills	How does a model sustain multi-step action?
XII	Multi-agent systems	Delegation, coordination, debate, specialization, emergence	How do multiple intelligences cooperate or conflict?
XIII	Orchestration	Routing, model selection, epistemic typing, regime selection	Who decides which intelligence should solve which problem?
XIV	Autonomous systems	Persistent goals, permissions, monitoring, recovery	How does the architecture operate continuously without human micromanagement?

Block	Theme	Core material	Architectural question
XV	Governance and safety	Verification, veto layers, uncertainty, auditability, containment	What may the system decide, recommend, and execute?

What Students Should Be Able to Do

A student who completes such a course should be able to inspect an intelligent system and answer, for every component:

1. **Function:** Is this component perceiving, predicting, judging, optimizing, controlling, remembering, or governing?
2. **Model:** What mathematical or computational model performs that function?
3. **Epistemics:** Does its output represent a point estimate, probability, latent representation, semantic hypothesis, feasible solution, or executable command?
4. **Uncertainty:** How is uncertainty represented and propagated?
5. **Authority:** Can the component recommend, decide, veto, or execute?
6. **Interfaces:** What information enters and leaves the component?
7. **Failure:** What happens when it is wrong?
8. **Feedback:** How is performance observed and used to update future behavior?

This is a much higher standard than knowing how to prompt an agent framework.

The Central Educational Proposition

The LLM revolution should not cause traditional AI to disappear from the curriculum. It should force us to understand traditional AI more deeply.

A CNN is not important because convolution must remain dominant forever. It is important because perception and spatial representation remain fundamental. An LSTM is not important because recurrence must defeat attention. It is important because temporal dependence remains fundamental. A VAE is not important because every system needs a VAE. It is important because latent-variable representation and probabilistic generation remain fundamental. Optimization is not “pre-AI mathematics.” It is the formal machinery through which objectives, trade-offs, and constraints become decisions. Control is not an implementation detail. It is the discipline through which decisions interact with dynamic reality.

LLMs and reasoning models add something genuinely powerful: semantic generality, contextual interpretation, natural-language interfaces, flexible decomposition, and increasingly the capacity to orchestrate external capabilities. Tool-use research makes explicit that specialized external systems can outperform or complement the language model on functions such as arithmetic and factual lookup [8]. Agent-planning research likewise shows that planning, external modules, reflection, and memory are separable architectural problems [11].

The resulting proposition is:

Perception constructs the world;
 Judgment interprets the world;
 Optimization chooses within the world;
 Control acts upon the world;
 Learning updates the system from what happened.

A Formal Multidimensional Taxonomy of Artificial Intelligence

The previous sections motivate a stronger formalization. A useful taxonomy of artificial intelligence cannot be one-dimensional because commonly used labels refer to different ontological levels. “Transformer” names a model architecture; “reinforcement learning” names a learning and decision paradigm; “agent” names a system role; “autonomy” names a degree of operational independence; and “reasoning” describes a cognitive function. Treating these as mutually exclusive categories is a category error.

We therefore define an AI component c by a six-dimensional descriptor:

$$\mathcal{T}(c) = (F_c, M_c, P_c, E_c, R_c, A_c)$$

where F_c is its **functional role**; M_c its **model family**; P_c its **learning or decision paradigm**; E_c its **epistemic output**; R_c its **architectural role**; and A_c its **autonomy and authority**. The descriptor is analytic rather than ontological: the axes interact, but separating them prevents category errors and provides a common language for system design.

Axis 1: Functional role

Let the functional space be

$$\mathcal{F} = \{\text{perception, representation, prediction, judgment, decision, optimization, control, learning, memory, governance}\}$$

The functional axis is deliberately independent of model family. A transformer can perform perception, forecasting, language modeling, or policy generation. A Bayesian model can perform state estimation, prediction, or risk inference. An LLM can participate in judgment or planning, but its function is not determined simply by the fact that it is an LLM.

Axis 2: Model family

The model-family axis identifies the computational object implementing a function.

Table 15: Axis 2: major model families

Family	Core mathematical idea	Typical strengths	Typical weaknesses
Linear/generalized linear models	Parametric mapping with explicit functional form	Interpretability, estimation, calibration	Limited nonlinear representation
Tree/ensemble models	Recursive partitioning and aggregation	Tabular prediction, robustness, variable interactions	Weak native representation of unstructured data

Family	Core mathematical idea	Typical strengths	Typical weaknesses
Kernel methods	Implicit high-dimensional feature maps	Flexible nonlinear decision boundaries	Scaling and representation limitations
CNNs	Local receptive fields and weight sharing	Spatial perception and structured signals	Limited explicit global reasoning
RNN/LSTM/GRU	Recurrent state transition through time	Sequential dependence, compact memory	Long-range training challenges
Transformers	Attention over token, patch, or state sequences	Scalable context integration and transfer	Compute intensity; not inherently grounded
Autoencoders/VAEs	Learned latent representations and reconstruction/probabilistic latent variables	Compression, representation, generation	Latents may lack causal meaning
Diffusion/generative models	Iterative denoising or generative density modeling	High-quality generation, scenario synthesis	Sampling cost, controllability
Probabilistic graphical models	Explicit factorization of joint probability	Interpretable uncertainty and dependence	Modeling complexity
Gaussian processes	Bayesian nonparametric function estimation	Calibrated uncertainty in small-data regimes	Poor scaling in large problems
State-space models	Latent dynamic state plus observation process	Filtering, dynamics, temporal structure	Model assumptions matter strongly
LLMs	Large autoregressive language models	Semantic generality, language interface, broad knowledge	Hallucination, calibration, grounding
Large reasoning models	Foundation models optimized for extended deliberation/test-time computation	Hard reasoning, decomposition, verification attempts	Latency, cost, variable reliability [15]
Vision-language/multimodal models	Shared representation across modalities	Cross-modal semantics and contextual interpretation	Safety-critical grounding remains difficult
World models	Learned transition/generative model of environment	Counterfactual prediction and planning support	Model bias and compounding imaginary error [13, 16]
Value/policy networks	Learned value functions or policies	Sequential control and decision learning	Reward/design and distribution-shift sensitivity
Optimization models	Explicit objective plus feasible set	Transparent trade-offs and constraints	Requires formulation and reliable inputs
Control models	Dynamic system plus feedback law	Stability, precision, low latency	Local validity and state-estimation dependence
Knowledge graphs/ontologies	Explicit entities and typed relations	Structured memory and logical relationships	Maintenance and schema brittleness

Table 14: Axis 1: functional roles of intelligence

Function	Canonical question	Typical output
Perception	What is observed?	Detected objects, labels, segments, features
Representation	What latent state best summarizes observations?	Embeddings, latent states, state vectors
Prediction	What may happen next?	Trajectories, distributions, scenarios
Judgment	What does this situation mean?	Regime, semantic hypothesis, priority, interpretation
Decision	Which alternative should be selected?	Action class, policy choice, intervention
Optimization	Which feasible choice best satisfies objectives?	Optimal/near-optimal solution and objective value
Control	How is the selected action executed?	Control signal, command, feedback response
Learning	How should parameters or policies change?	Updated weights, beliefs, policy, model
Memory	What past information should persist?	Retrieved episodes, summaries, state history
Governance	What is permitted, auditable, or vetoed?	Authorization, rejection, escalation, fallback

Axis 3: Learning and decision paradigm

The paradigm axis describes how capabilities are acquired or how decisions are derived.

Axis 4: Epistemic output

Two components may return data in the same software format while making fundamentally different kinds of claims. We therefore define an epistemic type:

$$\mathcal{E} = \{\text{measurement, estimate, distribution, representation, forecast, semantic hypothesis, counterfactual, solution}\}$$

This axis is crucial for orchestration. An LLM-generated semantic hypothesis and a calibrated posterior probability should not be consumed as if they were epistemically equivalent merely because both can be serialized as text or JSON.

Axis 5: Architectural role

The architectural role specifies how a component participates in a system:

$$\mathcal{R} = \{\text{specialist, router, orchestrator, critic, simulator, planner, executor, monitor, memory, safety-veto}\}.$$

An important implication follows: *an LLM is not inherently an orchestrator*. It becomes one only when assigned an orchestration role. Equally, an orchestrator need not be an LLM; routing and coordination can be rule-based, optimization-based, learned, or hybrid.

Axis 6: Autonomy and authority

Autonomy should be separated from model capability. We define an autonomy level α and an authority level β :

$$\alpha \in \{0, 1, 2, 3, 4, 5\}, \quad \beta \in \{\text{observe, recommend, decide, execute, veto}\}.$$

Table 16: Axis 3: learning and decision paradigms

Paradigm	Signal / mechanism	Canonical objective
Supervised learning	Labeled input-output examples	Minimize prediction loss
Unsupervised learning	Structure without labels	Discover latent organization or density
Self-supervised learning	Labels induced from the data itself	Learn transferable representations
Generative learning	Estimate or generate a data distribution	Model $p(x)$ or $p(x z)$
Imitation learning	Demonstrations from expert behavior	Match expert policy/distribution
Reinforcement learning	Reward from interaction	Maximize expected return
Bayesian learning	Prior plus likelihood	Update posterior beliefs
Online/continual learning	Sequential observations	Adapt while limiting forgetting
Search/planning	Explicit state/action exploration	Find a path or plan satisfying an objective
Mathematical optimization	Objective and constraints	Solve arg min / arg max over a feasible set
Inference-time reasoning	Additional computation at use time	Improve solution quality through deliberation
Evolutionary/meta-learning	Population search or learning-to-learn	Adapt strategies or the optimization procedure
Rule-based reasoning	Explicit symbolic conditions	Enforce known logic or policy

Capability and authority can therefore be intentionally mismatched. A powerful reasoning model may operate only at recommendation authority, while a simple certified safety controller has veto power.

The AI Architecture Tensor

The six axes can be represented as an architecture tensor:

$$\mathbb{A} = \mathcal{F} \times \mathcal{M} \times \mathcal{P} \times \mathcal{E} \times \mathcal{R} \times \mathcal{A}.$$

A concrete component occupies a point or small region of this tensor. For example:

Pedestrian CNN = (Perception, CNN, Supervised, Probability, Specialist, Reactive/Recommend),

DriveGPT = (Judgment, LLM/LRM, Self-supervised+reasoning, Semantic hypothesis, Orchestrator, Go

Trajectory optimizer = (Optimization, Mathematical program, Optimization, Solution, Planner, Reactive/Decide)

Safety controller = (Control/Governance, Control/rules, Deterministic, Permission/command, Safety-veto, Re

The tensor formulation prevents a single scalar ranking such as “more advanced AI.” Components may be more sophisticated on one axis and intentionally simpler on another.

The Function–Model Matrix

A useful educational device is to distinguish *can perform* from *comparative advantage*.

Table 17: Axis 4: epistemic types and their interpretation

Output type	Example	Interpretive requirement
Estimate	“Vehicle speed = 14.2 m/s”	Error variance and measurement assumptions
Probability distribution	“ $P(\text{cross}) = 0.34$ ”	Calibration, conditioning set, tail behavior
Latent representation	768-dimensional scene embedding	Meaning is implicit and task dependent
Forecast	Three possible trajectories	Horizon, scenario probabilities, model error
Semantic hypothesis	“Officer is directing traffic”	Grounding, evidence, confidence, alternatives
Counterfactual	“If we brake now, gap remains 4.8 m”	Simulator/world-model validity
Optimization solution	Trajectory τ^*	Objective, constraints, feasibility, robustness
Policy	Mapping from states to actions	Domain of validity and stability under shift
Control command	Brake pressure or steering command	Timing, safety envelope, actuator response
Permission/veto	“Action rejected”	Rule provenance and authority

Table 18: Axis 5: architectural roles

Role	System responsibility	Possible implementation
Specialist	Solve one bounded problem	CNN, Bayesian model, optimizer, LLM
Router	Select among capabilities	Classifier, rules, LLM, learned gate
Orchestrator	Decompose, coordinate, synthesize	LLM/LRM, planner, workflow engine
Critic/verifier	Challenge or validate output	Rules, second model, formal checker
World-model simulator	Generate futures or counterfactuals	Learned dynamics, simulator, diffusion model
Planner	Construct a sequence of actions	Search, LLM plan, MPC, task planner
Executor	Perform an authorized operation	Controller, API, deterministic service
Monitor	Detect anomalies and drift	Statistical detector, learned anomaly model
Memory	Preserve and retrieve state	Database, vector store, knowledge graph, episodic memory
Safety veto	Enforce non-negotiable envelope	Formal constraints, deterministic guard, certified controller

The matrix is deliberately non-exclusive. Modern multimodal foundation models increasingly participate in perception, while learned world models increasingly support planning. Surveys of autonomous-driving world models explicitly organize research around future-world generation, behavior planning, and interaction between prediction and planning [13, 16]. The taxonomy should therefore be interpreted as architectural guidance, not immutable boundaries.

The Paradigm–Architecture Matrix

The next matrix asks which learning and decision paradigms commonly support which system roles.

This matrix makes a central point: *orchestration is not itself a model family*. It is a system function that may be implemented through inference-time reasoning, search, optimization, learned routing, rules, or mixtures thereof.

A Formal Architecture for Hybrid Autonomous Intelligence

Table 19: Axis 6A: autonomy levels

Level	Name	Definition
0	Passive model	Produces output only when called; no persistent state
1	Reactive capability	Responds to current state; no independent planning
2	Stateful agent	Maintains memory or context across steps
3	Goal-directed agent	Plans and invokes tools toward specified goals
4	Adaptive autonomous system	Replans under changing conditions and monitors outcomes
5	Open-ended autonomous architecture	Sustains goals, model selection, learning, and coordination over long horizons subject to governance

Table 20: Axis 6B: authority levels

Authority	Meaning
Observe	May inspect state or data but not recommend action
Recommend	May propose an interpretation or action
Decide	May select an action or policy subject to external execution
Execute	May cause a real-world or system state change
Veto	May block actions proposed by other components

Let

$$\mathcal{S}_t = (B_t, W_t, M_t, G_t, C_t, \Gamma_t)$$

where B_t is the belief state inferred from perception, W_t a world model or predictive distribution, M_t persistent memory, G_t goals, C_t constraints and governance, and Γ_t the available library of models, tools, and agents.

An orchestrator Ω performs

$$\Omega : (\mathcal{S}_t, \Gamma_t) \mapsto (Q_t, \Gamma_t^*, J_t),$$

where Q_t is a set of subproblems, $\Gamma_t^* \subseteq \Gamma_t$ the selected capabilities, and J_t the interpreted decision regime.

Each selected capability γ_i returns

$$z_i = \gamma_i(q_i; \theta_i),$$

with epistemic type $E(z_i) \in \mathcal{E}$. A synthesis operator combines compatible outputs:

$$Z_t = \Sigma(z_1, \dots, z_n \mid E(z_1), \dots, E(z_n)).$$

The decision layer then solves

$$a_t^* = \arg \min_{a \in \mathcal{A}} L(a \mid Z_t, J_t, G_t)$$

subject to governance and safety constraints

$$a_t \in \mathcal{C}_{\text{safe}}(C_t).$$

Finally, the controller executes

$$u_t = K(a_t^*, x_t)$$

and feedback updates beliefs and memory:

$$(\mathcal{S}_{t+1}, M_{t+1}) = \Phi(\mathcal{S}_t, u_t, y_{t+1}).$$

Table 21: Indicative function–model matrix: H = strong comparative fit, M = useful, L = limited or auxiliary

	Percep.	Repr.	Predict	Judge	Optimize	Control	Learn	Memory	Govern
Classical ML	H	M	H	L	L	L	H	L	L
CNN/ViT	H	H	M	L	L	L	H	L	L
AE/VAE	M	H	M	L	L	L	H	L	L
RNN/LSTM/SSM	M	M	H	L	M	L	H	M	L
Bayesian models	M	M	H	M	M	L	H	M	M
World models	M	H	H	M	M	L	H	M	L
LLM	L–M	H	M	H	M	L	M	H	M
Large reasoning model	L–M	H	M	H	M–H	L	M	H	M
RL policy/value	M	M	H	M	H	M	H	M	L
Optimization/OR	L	L	M	L	H	M	L	L	M
Control	L	L	M	L	M	H	M	L	H
Knowledge graph/RAG	L	H	L	H	L	L	M	H	M
Rules/formal methods	L	L	L	M	M	H	L	L	H

Table 22: Paradigm–architecture matrix

	Specialist	Router	Orchestrator	Planner	Critic	Simulator	Executor	Safety
Supervised learning	H	H	L	L	M	L	L	L
Self-supervised learning	H	M	M	L	L	M	L	L
Bayesian inference	H	M	M	M	H	M	L	M
Generative modeling	H	L	M	M	L	H	L	L
Reinforcement learning	H	M	M	H	M	M	H	L
Search/planning	M	M	H	H	M	M	M	M
Optimization	H	M	M	H	H	M	H	H
Inference-time reasoning	M	H	H	H	H	M	L	L
Rule/formal reasoning	M	H	M	H	H	L	H	H

This formulation separates six operations that are often collapsed inside the word “agent”:

decompose → select models → infer → synthesize → optimize → execute.

Judgment as Regime Selection and Objective Formation

Let $\mathcal{Q} = \{q_1, \dots, q_K\}$ be a set of validated operational regimes. For autonomous driving these might include normal, defensive, construction, emergency, and comfort regimes.

Judgment may select a discrete regime:

$$q_t^* = \arg \max_{q_k} P(q_k \mid B_t, W_t, M_t, G_t),$$

or a mixture

$$\omega_t = (\omega_{1t}, \dots, \omega_{Kt}), \quad \sum_k \omega_{kt} = 1.$$

The active objective becomes

$$L_t(a) = \sum_{k=1}^K \omega_{kt} L_k(a).$$

Thus judgment can be interpreted as *objective formation and regime selection*, while optimization remains responsible for solving the resulting constrained decision problem. This captures the intuition that human-like judgment often changes what we care about before it changes the physical action itself.

Perception–Judgment–Optimization as an Intelligence Cycle

The triad can now be expressed as a cycle of transformations:

$$\underbrace{Y_t \rightarrow B_t}_{\text{Perception}} \rightarrow \underbrace{(B_t, W_t, G_t, M_t) \rightarrow J_t}_{\text{Judgment}} \rightarrow \underbrace{(J_t, G_t, C_t) \rightarrow a_t^*}_{\text{Optimization}} \rightarrow \underbrace{a_t^* \rightarrow u_t \rightarrow Y_{t+1}}_{\text{Control/Feedback}}.$$

Each transformation solves a different problem:

1. **Perception is epistemic inference:** estimate what is true.
2. **Judgment is semantic and normative inference:** determine what the situation means and which objectives matter.
3. **Optimization is prescriptive inference:** determine the best feasible action.
4. **Control is dynamical execution:** make the chosen action occur robustly in the real system.

This provides a disciplined account of why perception, judgment, and optimization jointly constitute operational intelligence. They correspond respectively to descriptive, interpretive, and prescriptive dimensions of intelligent action.

A Model-Portfolio View of Artificial Intelligence

An advanced autonomous system can be understood as holding a portfolio of models:

$$\Gamma = \{\gamma_1, \gamma_2, \dots, \gamma_N\}.$$

The orchestrator faces a model-selection problem:

$$\Gamma_t^* = \arg \max_{\Gamma' \subseteq \Gamma} [V(\Gamma' \mid q_t) - \lambda C(\Gamma') - \mu R(\Gamma')],$$

where V is expected informational or decision value, C computational and latency cost, and R risk.

This introduces a useful analogy: model orchestration is itself an optimization problem under uncertainty. A reasoning model may be powerful but slow. A calibrated statistical model may be narrow but reliable. A deterministic controller may be cognitively simple but safety critical. The system should optimize not merely answers, but also *which intelligence to consult*.

Implications for Curriculum Design

The multidimensional taxonomy suggests that a rigorous course should require students to classify every model using a common template.

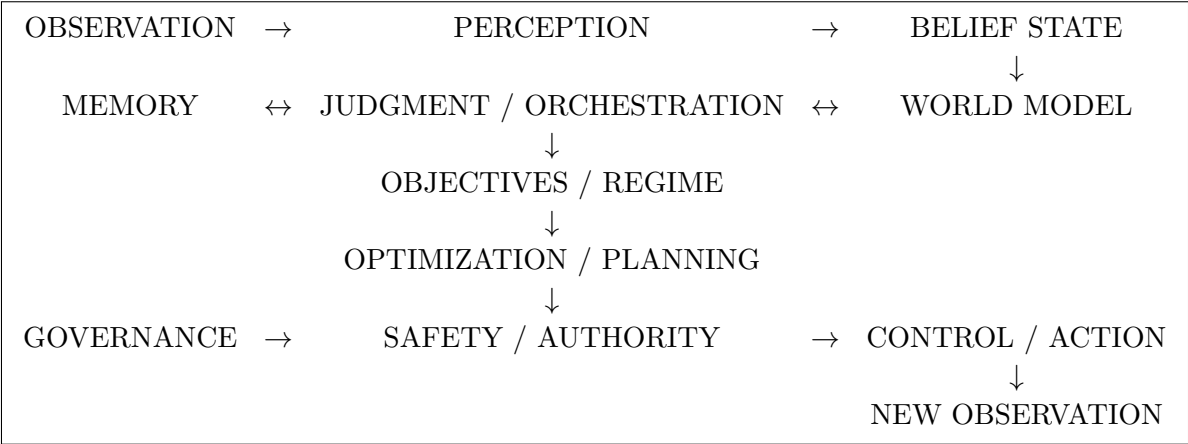
The final educational objective is therefore not merely to build an agent. It is to design a *typed architecture of intelligence*.

A Compact Synthesis: The Architecture of Intelligence

Table 23: Required model card for a Modeling Artificial Intelligence curriculum

Field	Student must specify
Function	Perception, representation, prediction, judgment, decision, optimization, control, learning, memory, governance
Model family	Statistical, neural, probabilistic, foundation, reasoning, world, RL, optimization, control, symbolic, knowledge
Paradigm	Supervised, self-supervised, Bayesian, generative, RL, search, optimization, reasoning, etc.
Epistemic output	Estimate, distribution, representation, semantic hypothesis, counterfactual, solution, command
Architectural role	Specialist, router, orchestrator, critic, planner, executor, monitor, memory, safety veto
Autonomy level	Passive through open-ended autonomous
Authority	Observe, recommend, decide, execute, veto
Failure mode	Distribution shift, hallucination, miscalibration, infeasibility, instability, reward misspecification, etc.
Verification method	Calibration, formal constraints, cross-model checking, simulation, human approval, runtime monitoring

The complete architecture can be summarized as



No single model need occupy the center permanently. The architecture can contain specialized ML models, probabilistic estimators, foundation models, reasoning models, world models, optimization engines, controllers, agents, and hard constraints. The intellectual object of study is therefore not the individual model but the *composition of heterogeneous intelligences*.

Seven Principles of Model-Aware Artificial Intelligence

The analysis can be condensed into seven principles.

1. **Function precedes model.** Begin with the computational function required, not with a favored architecture.
2. **No model is the field.** LLMs, CNNs, world models, optimizers, and controllers occupy different locations in the architecture.
3. **Prediction is not judgment.** A forecast does not determine which objective, regime, or risk tolerance should govern.

4. **Judgment is not execution.** Semantic competence should not automatically imply operational authority.
5. **Orchestration is a model-selection problem.** Mature agents must reason about the comparative advantages and epistemic properties of the capabilities they invoke.
6. **Autonomy is architectural.** Persistence, feedback, memory, authority, monitoring, and recovery matter as much as the underlying foundation model.
7. **Safety is a structural property.** It should be created by constraints, verification, authority design, redundancy, and feedback—not merely by asking a powerful model to behave safely.

Conclusion

The most consequential mistake in contemporary AI education is not enthusiasm for LLMs; it is a loss of architectural perspective. The visibility of the LLM has made one layer of intelligence appear to be the whole system.

The deeper architecture is heterogeneous. Intelligence emerges from a chain of functions that include perception, representation, prediction, uncertainty, judgment, optimization, action, feedback, and learning. Different model families possess different comparative advantages across those functions.

Multi-agent architectures have already moved toward this insight, but the common vocabulary of “agents, tools, and skills” is too coarse. It tells students how software components are packaged without necessarily teaching what forms of intelligence they embody.

The next step is **model-aware orchestration**: architectures in which an orchestrator understands whether it is consulting a classifier, probabilistic estimator, world model, reasoning model, optimizer, simulator, controller, or safety system; understands the epistemic status of their outputs; and respects explicit boundaries on authority.

The pedagogical objective should therefore move beyond model literacy and beyond prompt literacy toward **architectural intelligence**.

The graduate of a serious AI course should not merely know how to use an LLM. The student should be able to answer:

What intelligence is required here, what model supplies it, and how should the pieces cooperate?

That is the foundation for modeling artificial intelligence as a field rather than as a single technology.

References

- [1] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, 521, 436–444, 2015.
- [2] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, 2017.
- [3] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014.
- [4] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, 518, 529–533, 2015.

- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021.
- [7] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” ICLR, 2023.
- [8] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” NeurIPS, 2023.
- [9] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” arXiv:2308.08155, 2023.
- [10] J. S. Park et al., “Generative Agents: Interactive Simulacra of Human Behavior,” UIST, 2023.
- [11] X. Huang et al., “Understanding the Planning of LLM Agents: A Survey,” arXiv:2402.02716, 2024.
- [12] T. Xie et al., “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments,” arXiv:2404.07972, 2024.
- [13] T. Feng, W. Wang, and Y. Yang, “A Survey of World Models for Autonomous Driving,” arXiv:2501.11260, 2025.
- [14] Y. Gao et al., “Foundation Models in Autonomous Driving: A Survey on Scenario Generation and Scenario Analysis,” arXiv:2506.11526, 2025.
- [15] X. Qu et al., “A Survey of Efficient Reasoning for Large Reasoning Models: Language, Multi-modality, and Beyond,” arXiv:2503.21614, 2025.
- [16] S. Tu et al., “The Role of World Models in Shaping Autonomous Driving: A Comprehensive Survey,” arXiv:2502.10498, 2025.
- [17] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.